

AULA 2.1 TMS

ATIVIDADE:: GERANDO VERSÕES (1)

Imagine que esteja iniciando um novo projeto de software:

1. Qual seria sua estratégia para identificar as versões? Justifique.

Minha estratégia seria utilizar o **Versionamento Semântico (SemVer)**, representado pelo formato:

MAJOR.MINOR.PATCH

Essa estratégia facilita a identificação do tipo de mudança realizada no software. Dessa forma:

- **MAJOR:** indica mudanças grandes e incompatíveis com versões anteriores;
- **MINOR:** representa a adição de novas funcionalidades compatíveis com a versão atual;
- **PATCH:** identifica correções de erros e pequenos ajustes que não alteram o funcionamento esperado.

Utilizar esse padrão torna o histórico do projeto mais organizado e permite que os usuários compreendam rapidamente o impacto de cada atualização.

2. Como nomearia a primeira versão para o público?

A primeira versão estável disponibilizada para o público seria: v1.0.0

Justificativa: o número 1 indica o primeiro lançamento oficial do produto, enquanto os campos MINOR e PATCH iniciam em zero por não haver ainda funcionalidades adicionais ou correções após o lançamento.

3. Após liberado o projeto, uma nova funcionalidade foi requisitada e implementada.

Como nomearia esta nova versão?

Como houve apenas a adição de uma funcionalidade compatível com a versão anterior, a nova versão seria: v1.1.0

O incremento ocorre no campo **MINOR**, pois não houve quebra de compatibilidade

4. Considerando a sequência anterior com o esquema de versionamento SemVer.

Como ficaria o histórico de versões?. O histórico seria: v1.0.0 → v1.1.0 Onde:

- **v1.0.0:** primeira versão pública;
- **v1.1.0:** primeira atualização com nova funcionalidade compatível.

5. A partir da versão indicada antes, uma série de 3 correções, seguida por duas funcionalidades compatíveis com a versão atual e mais outras 2 correções foram publicadas em série. Qual seria a versão mais recente? Partindo da versão: v1.1.0

Temos:

- 3 correções:
 - v1.1.1
 -
 -
- 2 novas funcionalidades:
 - v1.1.2
 - v1.1.3
- 2 novas funcionalidades:
 - v1.2.0
 -

v

1.3.0

- 2 novas correções:
 - v1.3.1
 -

v

1.3.2

Portanto, a versão mais recente seria: v1.3.2

6. Uma versão nova exigiu mudanças críticas na API, foi lançada e na sequências houveram três novas versões: uma com adição de funcionalidades, seguidas de 2 com correções de bugs. Como fica o histórico?

Partindo da versão anterior (**v1.3.2**):

- Mudança crítica na API: ◦ v2.0.0
- Nova funcionalidade:
 - v2.1.0
- Correção de bug: ◦ v2.1.1
- Nova correção: ◦ v2.1.2 Assim, o histórico fica v1.3.2 ↓ v2.0.0 ↓ v2.1.0 ↓ v2.1.1 ↓ v2.1.2

A alteração da API exigiu a mudança do número **MAJOR**, pois houve quebra de compatibilidade

7. Pesquise exemplos representativos de versões reais de software considerando cada um dos termos apontados.

Termo SemVer	Significado	Exemplo Real
MAJOR	Mudanças incompatíveis	Python: 2.x → 3.0

MINOR	Novas funcionalidades compatíveis	Node.js: 18.19.0 → 18.20.0
PATCH	Correções de bugs	Google Chrome: 137.0.7151.69 → 137.0.7151.70

Análise:

- O **Python 3.0** trouxe diversas mudanças incompatíveis em relação ao Python 2, exigindo adaptações nos códigos existentes.
- O **Node.js** frequentemente adiciona novos recursos mantendo compatibilidade dentro da mesma versão principal.
- O **Google Chrome** publica atualizações PATCH com frequência para corrigir falhas e melhorar a segurança sem alterar funcionalidades importantes.

ATIVIDADE:: IGNORANDO ARQUIVOS 1.

Descreva o resultado esperado para um repositório com um arquivo .gitignore contendo:

```
!lib.a
/TODO
build/
doc/*.txt
doc/**/*.pdf
```

O arquivo .gitignore define quais arquivos ou diretórios não serão monitorados pelo Git. No exemplo apresentado, o arquivo lib.a será explicitamente acompanhado pelo Git; o arquivo TODO localizado na raiz será ignorado; toda a pasta build será desconsiderada; arquivos .txt diretamente dentro de doc serão ignorados; e arquivos .pdf presentes em qualquer subpasta de doc também não serão monitorados.

Conclusão

O Versionamento Semântico é uma prática essencial no desenvolvimento de software, pois padroniza a identificação das versões e comunica claramente aos usuários e desenvolvedores o impacto de cada atualização. Por meio dele, torna-se mais fácil acompanhar a evolução do projeto, planejar atualizações e manter a compatibilidade entre diferentes versões do sistema.
